MOJO · VULKAN · ADRENO 840 · QUALCOMM’S STOCK DRIVER

Mojo on the Adreno 840

A native Mojo target for the Snapdragon 8 Elite Gen 5’s GPU, its matrix units, and three language models running on Qualcomm’s own Vulkan driver.

Galaxy S26 Ultra (Samsung Remote Test Lab) · Qualcomm SM8850 reference phone (Qualcomm Device Cloud) · October 2026

The GPU, its driver, and the phones

Adreno 840

The GPU of the Snapdragon 8 Elite Gen 5 (SM8850), with matrix units for 64-row tiles.

The stock driver

Qualcomm’s Vulkan 1.4 driver as phones ship it: 512.842.19 and 512.842.25.

Two remote labs

A Galaxy S26 Ultra (Samsung) and Qualcomm’s SM8850 reference phone, both over adb.

For compute, the driver has	On the Adreno 840
Cooperative matrices	VK_KHR_cooperative_matrix: tiles 64 rows tall, 16, 32 or 64 wide
Matrix products	fp16 into fp16 (K 16), fp32 into fp32 (K 8), int8 into int32 (K 32); not fp16 into fp32
Subgroups, shared memory	64 lanes; 32 KB a workgroup
Not there	VK_QCOM_cooperative_matrix_conversion, on both driver builds

A native adreno840 target in Mojo’s compiler

Mojo
a GPU kernel, as for any GPU

Target plugins
64-lane warps, barriers, math, matrices

Vulkan ABI
buffers, params, matrix types

SPIR-V 1.6
LLVM’s backend, patched for matrices

C on Vulkan
one small runner per model

One flag

--target-accelerator adreno840 compiles any Mojo GPU kernel to a Vulkan compute shader. Pointer arguments become storage buffers, numbers become slots of a params buffer.

No MAX runtime on Android

So each model is a few hundred lines of C: load the weights, record every dispatch into one command buffer, check the results.

The matrix units, from Mojo

```
acc = CoopMatrix[DType.float16, 64, 64, ACCUMULATOR].zero()
for k in range(0, K, 16):
    a_tile = CoopMatrix[DType.float16, 64, 16, MATRIX_A].load(a + row * K + k, K)
    b_tile = CoopMatrix[DType.float16, 16, 64, MATRIX_B].load(b + k * N + col, N)
    acc = a_tile.multiply_add(b_tile, acc)
acc.store(c + row * N + col, N)
```

3.7 TFLOP/s

fp16 matmul, 2048^3, tiles staged in shared memory

2.5 TFLOP/s

fp32, same size: fp32 sums, at two thirds the speed

3/3

MAX’s matmul graphs pass on the phone, 8× its naive fp16 kernel

What the stock driver needed

Both builds

fp16 in shared memory sits 4 bytes apart

Stage fp16 tiles with matrix loads and stores, offsets halved.

Both builds

A sum loaded from memory breaks the pipeline

Start every sum at zero; add the residual in its own kernel.

Both builds

All 32 KB of shared memory: wrong results

Stay at 16 KB: K steps of 16.

Android 17 build

Same version number, rejects our matrices

The compiler now writes zero tiles as constants.

Android 16 build

Fused attention fails to link

The runner falls back to per-row attention.

The hardware’s shapes

fp16 products sum only in fp16

Long sums use fp32 tiles, at two thirds the speed.

> Each one either failed outright or silently gave wrong numbers, so every run checks every output.

Mojo on the Adreno 840 · 5 · driver 512.842.19 (Android 16 and 17) and 512.842.25 · fixes in the target and the runners

LLAMA 3.2 1B ON THE ADRENO 840: GENERATING TEXT

43 ms

per token with fp16 weights, 2.47 GB read for every token

13 ms

per token with 4-bit weights, an fp16 scale per 64

48/48

tokens identical to PyTorch, for both weight formats

> Generating reads every weight once per token, so memory bandwidth sets the pace: 4-bit weights read a quarter of the bytes.

Mojo on the Adreno 840 · 6 · llama/ · gemv, gemv_q4: one 64-lane subgroup per row · Galaxy S26 Ultra, Android 16

LLAMA 3.2 1B: THE PROMPT ON THE MATRIX UNITS



28×

faster prompt processing, same phone and driver

48/48

tokens identical to PyTorch, logits within 6.0e-5

46 ms

attention for all 16 layers, both products on the matrix units

> Each projection is one matrix product for all 512 rows: fp32 tiles, the fp16 weights converted as they are staged in shared memory, 1.8 TFLOP/s.

Mojo on the Adreno 840 · 7 · llama/kernels/prefill · Galaxy S26 Ultra, Android 17 · prompt: llama/prompts/long.txt

BONSAI 2 2B: SOMEONE ELSE'S MOJO KERNELS, UNCHANGED

`kernels.mojo`
from mav3ri3k/modular's Android port, written for its OpenCL target

→

`adreno840`
our target: all 21 kernels to Vulkan SPIR-V, not one line changed

→

`bonsai_vk.c`
the port's forward pass in C: 721 dispatches a token, one command buffer

80 tok/s
decode, 12.4 ms a token; the port reports about 75 through OpenCL on an S26

6×
faster prompt: 412 tokens in 0.87 s instead of 5.2 s, on the matrix units

16/16
greedy tokens identical to the FP32 reference, token by token or batched

> For the prompt, one dispatch runs a decode kernel on every row. New: the ternary projections and attention scores on cooperative matrices, the DeltaNet recurrence as one scan.

Mojo on the Adreno 840 · 8 · bonsai/ · decode: Galaxy S26 Ultra · prefill: Qualcomm's SM8850 reference phone

TERNARY QWEN3-8B ON THE ADRENO 840

25 tok/s

for an 8B model: 40 ms a token, 2.19 GB of weights read each time

16/16

greedy tokens identical to MLX, every one of 151,669 logits within 0.0082

7

small new Mojo kernels; the ternary projections are Bonsai's, unchanged

> "The sky appears blue because sunlight, which contains all colors of the spectrum, is scattered more efficiently by the atmosphere's molecules..."

Mojo on the Adreno 840 · 9 · qwen3/ · prism-ml/Ternary-Bonsai-8B-mlx-2bit · Qualcomm's SM8850 reference phone

Three models on one phone GPU

Model	On the GPU	Generating	Long prompt	Checked against
Llama 3.2 1B, fp16	2.47 GB	43 ms a token	431 tokens: 0.63 s	PyTorch: 48/48
Llama 3.2 1B, 4-bit	1.18 GB	13 ms a token	431 tokens: 0.62 s	PyTorch: 48/48
Bonsai 2 2B, ternary	0.48 GB	12.4 ms a token	412 tokens: 0.87 s	MLX FP32: 16/16
Qwen3-8B, ternary	2.19 GB	40 ms a token	one token at a time	MLX FP32: 16/16

Generating: bandwidth

One token is a matrix-vector product: the time is the bytes read. Fewer bits per weight is the lever.

Prompts: matrix units

All tokens at once reuse every weight: 6 to 28 times faster than token by token on the same phone.

Mojo on the Adreno 840 · 10 · Llama: Galaxy S26 Ultra · Bonsai: decode S26 Ultra, prompt reference phone · Qwen3: reference phone

The same models, run from Mojo

```
model.mojo (DeviceContext launches) → MAX’s HAL DeviceContext → libM_driver_vulkan.so → Vulkan
```

Model, its forward pass in Mojo	Mojo DeviceContext	C runner	Checked
Llama 3.2 1B, fp16	**40.6 ms** a token	43 ms	PyTorch: 48/48
Llama 3.2 1B, 4-bit	**12.8 ms** a token	13 ms	PyTorch: 48/48
Bonsai 2 2B: the port’s own model.mojo	**13.2–13.5 ms** a token	12.4 ms	MLX FP32: 16/16
Qwen3-8B, ternary	**39.8 ms** a token	40 ms	MLX FP32: 16/16

MAX’s runtime is cross-built for Android, and our driver plugin implements MAX’s device interface on Vulkan. The phone downloads each checkpoint from Hugging Face and converts it itself.

> A launch costs 6.5 µs, 64 launches a submission: the forward pass in Mojo is as fast as the hand-written runners.

What VK_QCOM_cooperative_matrix_conversion would add

Weights straight into the matrix

One 32-bit word is 16 ternary codes, one fp16 K16 column: each lane builds its column in registers. No shared-memory staging, no barrier.

fp16 speed, fp32 accuracy

The 840 sums fp16 products only in fp16. Pull the accumulator out every 128 values and add it into fp32: 3.5 TFLOP/s fp16 instead of 1.8 fp32.

Attention without the round trip

Softmax between $Q \cdot K^T$ and $P \cdot V$ stays in registers, instead of the scores going through shared memory.

Integer products

Ternary weights are exact in int8: int8 products, the group scales applied in registers. Activations in int8 would cost accuracy.

Model	Today, KHR only (measured)	With conversion (estimate)
Llama 3.2 1B, 431-token prompt	0.63 s, products 0.56 s in fp32	**about 0.4 s**: fp16 products, fp32 sums
Bonsai 2 2B, 412-token prompt	0.87 s, products 0.80 s in fp32	**about 0.5 s**: no staging, fp16 products
Ternary Qwen3-8B	40 ms a token, decode only	decode unchanged; a matrix-unit prefill worth building

Mojo on the Adreno 840 · 12 · Vulkan 1.4.342 · not on the drivers we tested (512.842.19, .25) · runners load qcom/*.comp kernels when present

What the Adreno 840 taught us

The driver is the platform	Two builds with one version number behaved differently. Check every output.
Matrix units are for prompts	Generating is matrix-vector work, set by bandwidth; prompts gain 6 to 28 times.
fp16-only sums set the ceiling	Accurate prompts run on fp32 tiles at two thirds the speed.
Mojo kernels carry over	An OpenCL port’s kernels ran unchanged on Vulkan, and served a second model.
The runtime was the gap	With MAX’s runtime on Android and a Vulkan plugin, models run as Mojo host code.

What comes next

The conversion extension in Mojo

Its instructions in LLVM’s SPIR-V backend and our target, so the QCOM kernels become Mojo too.

Faster prompts

fp16 products with fp32 sums wherever a driver allows; a matrix-unit prefill for Qwen3-8B.

MAX graphs on the phone

Mojo DeviceContext code runs there now; next, MAX’s graph executor for Android, so graphs run directly.

> One target flag, the stock driver, three models checked token for token: the Adreno 840 runs Mojo today.